Few-Shot Local Cleaner Report

SinLlama Base Model — Few-Shot Prompting for Sinhala ASR Text Correction (No Fine-Tuning)

Version 1.0 · August 2026 · Prepared from SinLlama_Llama_3_8B_Merged_cleaner_few_shot.ipynb

Table of Contents

1. Overview 2
2. How This Approach Works (General Flow) 2
3. Environment and Requirements 2
4. Code Walkthrough 3
   - 4.1 Step 1 — Mount Google Drive & Set Model Cache 3
   - 4.2 Step 2 — Install / Upgrade bitsandbytes 3
   - 4.3 Step 3 — Load Base Model and Tokenizer (4-bit) 3
   - 4.4 Step 4 — Few-Shot Prompt Template & Cleaning Function 4
   - 4.5 Step 5 — Test Execution 4
5. Observed Output and Behaviour 5
6. Known Limitation: Repetition Loop 5
7. Few-Shot Prompting vs. Fine-Tuned (QLoRA) Approach 6
8. Summary 6

1. Overview

This report documents a local/Colab inference approach for cleaning noisy Sinhala ASR (Automatic Speech Recognition) transcripts using the SinLlama base model (SAWithanage/SinLlama-Llama-3-8B-Merged) with 4-bit quantization. Unlike the earlier CPU/GPU inference reports, this approach does not use a fine-tuned LoRA adapter at all. Instead, it relies purely on few-shot prompting: five example Noisy → Clean Sinhala text pairs are embedded directly in the prompt to condition the unmodified base model to perform the same correction task.

This report is based on the notebook SinLlama_Llama_3_8B_Merged_cleaner_few_shot.ipynb. The accompanying local_cleaner_few-shot.py file that was uploaded alongside it was empty at the time this report was generated, so all code, configuration, and output shown below is drawn entirely from the notebook's cells and their recorded execution output.

2. How This Approach Works (General Flow)

The notebook implements a five-stage flow. The key difference from the fine-tuned workflows documented previously is that stages 3 and 4 use in-context examples instead of trained adapter weights:

- Mount Google Drive and redirect the Hugging Face model cache there, so downloaded weights persist across Colab sessions.
- Install/upgrade the bitsandbytes library needed for 4-bit quantized loading.
- Load the unmodified SinLlama-Llama-3-8B-Merged base model and tokenizer directly from Hugging Face, quantized to 4-bit (no LoRA adapter is loaded).
- Build a fixed prompt containing five hand-written Noisy/Clean Sinhala example pairs, append the new noisy input text, and ask the model to complete the “Clean:” line.
- Generate a completion, strip the prompt back off the model's output, and cut the result at the first newline so the model doesn't keep generating extra invented dialogue.

3. Environment and Requirements

The notebook was written for Google Colab and mounts Google Drive to cache the downloaded model between sessions.

Component	Configuration / Requirement	Why It's Needed
Execution environment	Google Colab (Google Drive mounted)	Persists the downloaded ~16 GB base model in Drive so it does not need to be re-downloaded every session.
GPU	CUDA-capable GPU (e.g., Colab T4)	Required — the model and inputs are explicitly moved to “cuda”; there is no CPU fallback path in this notebook.
Base model	SAWithanage/SinLlama-Llama-3-8B-Merged	Loaded directly with no LoRA adapter — the unmodified base model performs the correction using only prompt context.
Quantization	4-bit (NF4, double quantization, float16 compute)	Reduces VRAM usage via BitsAndBytesConfig, the same

Component	Configuration / Requirement	Why It's Needed
		technique used in the GPU inference report.
Key library	bitsandbytes ≥ 0.46.1	Explicitly upgraded in the notebook to support the quantization config used.
Prompting method	Few-shot (5 embedded example pairs)	Replaces fine-tuning — the model is steered purely through in-context examples in every call.

4. Code Walkthrough

4.1 Step 1 — Mount Google Drive & Set Model Cache

The first cell mounts the user's Google Drive and points the Hugging Face cache environment variables at a folder inside it, so the large base model is downloaded once and reused on future runs.

```
from google.colab import drive
import os

drive.mount('/content/drive')

cache_dir = '/content/drive/MyDrive/HF_Models_Cache'
os.makedirs(cache_dir, exist_ok=True)

os.environ['HF_HOME'] = cache_dir
os.environ['TRANSFORMERS_CACHE'] = cache_dir
```

Recorded output confirmed the cache location: "Hugging Face cache set to: /content/drive/MyDrive/HF_Models_Cache".

4.2 Step 2 — Install / Upgrade bitsandbytes

The second cell ensures a recent-enough version of bitsandbytes is available to support the 4-bit quantization configuration used in the next step.

```
!pip install -U bitsandbytes>=0.46.1
```

4.3 Step 3 — Load Base Model and Tokenizer (4-bit)

The base model is loaded directly from Hugging Face with a 4-bit BitsAndBytesConfig — the same nf4 / double-quantization / float16-compute pattern used in the GPU inference script from the earlier report. Notably, there is no PeftModel / LoRA loading step anywhere in this notebook.

```
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
import torch

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
```

```
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16
)

tokenizer = AutoTokenizer.from_pretrained("SAWithanage/SinLlama-Llama-3-8B-Merged")
model = AutoModelForCausalLM.from_pretrained(
    "SAWithanage/SinLlama-Llama-3-8B-Merged",
    quantization_config=bnb_config,
    device_map="auto"
)
```

4.4 Step 4 — Few-Shot Prompt Template & Cleaning Function

The clean_asr_text() function builds a single long prompt containing five worked Noisy → Clean Sinhala call-transcript examples, followed by the new "Noisy: {noisy_input}" line and an empty "Clean:" line for the model to complete. Because this is the raw base model (not instruction-tuned or fine-tuned), this few-shot "completion" framing is what steers it toward producing a correction rather than continuing the noisy text or writing something unrelated.

```
def clean_asr_text(noisy_input):
    # We use strict formatting to force the Base Model into completion mode
    prompt = f"""Noisy: <example 1 noisy Sinhala ASR transcript>
Clean: <example 1 corrected Sinhala text>

Noisy: <example 2 noisy Sinhala ASR transcript>
Clean: <example 2 corrected Sinhala text>

Noisy: <example 3 noisy Sinhala ASR transcript>
Clean: <example 3 corrected Sinhala text>

Noisy: <example 4 noisy Sinhala ASR transcript>
Clean: <example 4 corrected Sinhala text>

Noisy: <example 5 noisy Sinhala ASR transcript>
Clean: <example 5 corrected Sinhala text>

Noisy: {noisy_input}
Clean:"""
```

The notebook's actual prompt embeds five complete SLT Mobitel-style call-center transcripts (billing queries, package upgrades, fault reports, etc.) as the five examples; they are abbreviated above for readability, but the structure — five full Noisy/Clean pairs followed by the live input — matches the notebook exactly.

After the prompt is built, it is tokenized, sent to the GPU, and generated with low-temperature sampling:

```
    inputs = tokenizer(prompt, return_tensors="pt").to("cuda")

    outputs = model.generate(
        **inputs,
        max_new_tokens=512,
        temperature=0.1,
        do_sample=True,
        pad_token_id=tokenizer.eos_token_id,
    )
```

```
    full_output = tokenizer.decode(outputs[0], skip_special_tokens=True)

    # Slice off the original prompt text so we only have the new generation
    generated_text = full_output[len(prompt):].strip()

    # Base models might try to keep writing (e.g., a new "Noisy:" line).
    # We force it to stop by splitting at the first newline character.
    clean_output = generated_text.split('\n')[0].strip()

    return clean_output
```

- max_new_tokens=512 caps how long a single correction can be.
- temperature=0.1 with do_sample=True keeps generation close to deterministic while still technically sampling.
- The prompt text is sliced off the decoded output, since a base model's generate() call returns the prompt plus the completion together.
- The result is cut at the first newline as a manual stopping rule, because a raw base model has no fine-tuned sense of when a correction "ends" and may otherwise keep inventing further "Noisy:/Clean:" turns.

4.5 Step 5 — Test Execution

The final cell runs a single test call, passing a noisy Sinhala ASR transcript (a customer service call about a broken router/TV/phone line) into clean_asr_text() and printing the input and the model's output.

```
print("\n--- System Ready ---")
test_noisy_text = "<noisy Sinhala ASR transcript>"

print(f"INPUT: {test_noisy_text}")
cleaned = clean_asr_text(test_noisy_text)
print(f"CLEANED: {cleaned}")
```

5. Observed Output and Behaviour

The notebook's recorded execution output confirms the model loaded and ran successfully. A transformers warning was also recorded during generation:

```
--- System Ready ---
INPUT: <noisy Sinhala ASR transcript about a router / TV / phone fault>
CLEANED: <corrected Sinhala transcript, see Section 6>
```

Warning recorded in the notebook output: "Both max_new_tokens (=512) and max_length (=4096) seem to have been set. max_new_tokens will take precedence." This is an informational transformers notice, not an error — generation still completed and returned output.

6. Known Limitation: Repetition Loop

In the one recorded test run, the model correctly cleaned the opening portion of the transcript (greeting, TV/router fault description, confirming the phone line had no issue), but it then fell into a repetition loop: it repeated a short phrase giving a phone number roughly fifty times in a row before the 512-token generation limit was reached, rather than terminating naturally.

This is a documented behaviour of the actual notebook run, not a hypothetical concern:

- Because this is the unmodified base model steered only by five in-context examples, it has no fine-tuned stopping behaviour for the correction task — it can fall into degenerate repetition once it runs out of confident content to generate.
- The manual stopping rule (split on the first newline) does not help here, because the model kept repeating the same phrase on a single line rather than starting a new line.
- The max_new_tokens=512 cap is what eventually stopped generation — without it, the repetition could plausibly have continued indefinitely.
- Lowering max_new_tokens, adding a repetition penalty, or using stronger stopping criteria (e.g., stopping on a repeated n-gram) would likely reduce this behaviour, though none of these were present in the notebook as uploaded.

7. Few-Shot Prompting vs. Fine-Tuned (QLoRA) Approach

This notebook can be directly compared with the QLoRA fine-tuning workflow documented separately (SinLlama Training Mode Documentation). The two represent different ways of adapting the same base model to the same Sinhala ASR correction task.

Aspect	Few-Shot Cleaner (this notebook)	QLoRA Fine-Tuned Model
Model weights used	Unmodified base model	Base model + trained LoRA adapter merged in
How the task is taught	5 example pairs embedded in every prompt	126 training examples, learned via gradient updates
Setup cost	No training — write examples once	Requires a training run (2 fine-tuning passes recorded)
Per-call prompt size	Larger — carries all 5 examples every call	Small — only the instruction + input text
Stopping behaviour	Manual newline-split hack; can repeat/loop (see Section 6)	Trained to emit an EOS/eot token at the natural end of a correction
Observed reliability (this data)	Correct on early sentences, degenerated into a repetition loop	Evaluated formally with WER/CER/ROUGE metrics
Best suited for	Fast experimentation without a training pipeline	Production use once trained, given its more predictable stopping behaviour

8. Summary

- This notebook cleans noisy Sinhala ASR text using the unmodified SinLlama-Llama-3-8B-Merged base model, loaded in 4-bit, with no LoRA fine-tuning involved.
- Correction behaviour is induced entirely through a fixed few-shot prompt containing five worked Noisy/Clean example pairs, followed by the live input.
- The model's raw completion is post-processed in code: the prompt is sliced off, and the result is cut at the first newline as a manual stopping rule.

- The one recorded test run showed the model correcting the early part of a transcript correctly, then falling into a long repetition loop that only stopped because of the 512-token generation cap.
- Compared with the QLoRA fine-tuned workflow, this few-shot approach needs no training step but showed less reliable stopping behaviour in the recorded output.

***